#Hashlock.

Forte AUD (Wallet)

SMART CONTRACT

Security Audit

Platform

ETH

hashlock.com.au

May 2024

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

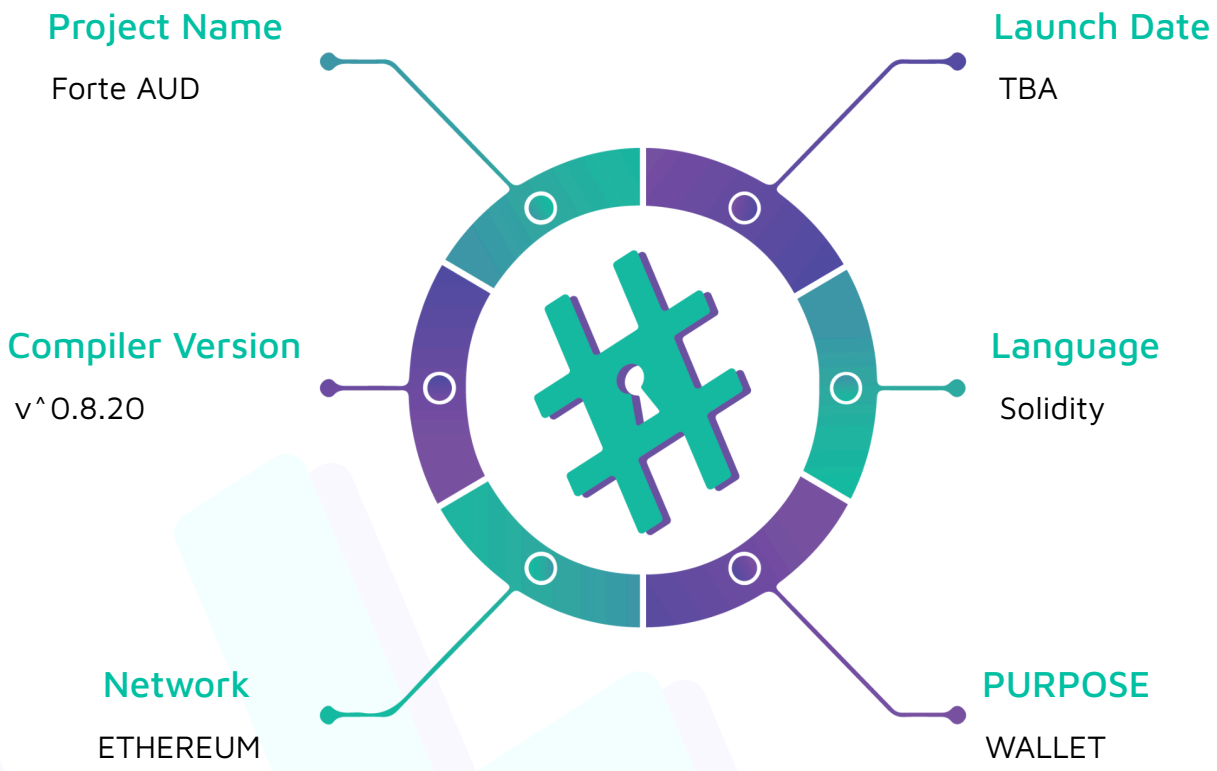
The Forte AUD team partnered with Hashlock to conduct a security audit of their smart contracts for Forte AUD (AUDF). Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Forte AUD is an AUD Stablecoin project. The project includes wider functionality and interoperability.

Project Name: Forte AUD
Compiler Version: ^0.8.20
Website: www.forteaud.com
Logo:

Visualised Context:



Project Visuals:

About AUDF

AUDF facilitates simple, fast, and cost-effective transactions across the globe granting uninterrupted access to payments and financial solutions.

Reserves

AUDF is backed and fully redeemable for Australia's sovereign currency.

Transparent

Published third party audit reports of AUDF reserves.

Efficient

Operate seamlessly without being constrained to traditional banking hours and borders.

Audit scope

We at Hashlock audited the solidity code within the Forte AUD project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	**Forte AUD Protocol Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**March, 2024**
Contract 1	Wallet.sol

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We identified some vulnerabilities that were to be addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

Hashlock found:

0 High severity vulnerabilities

1 Medium severity vulnerabilities

1 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Purpose

Claimed Behaviour	Actual Behaviour

Stablecoin.sol
- ERC20Upgradeable contracts with more functionality:
  - 2 types of mint()
  - 2 types of burn()
  - Blacklist() function
  - Unblacklist() function
	Contract achieves this functionality.

Wallet.sol
- Wallet contract:
  - Interact with Stablecoin.sol functionality: mint(), burn()
  - Add beneficiary
  - Remove Beneficiary
  - Increase time limit for beneficiary
  - Decrease time limit for beneficiary
	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the Forte AUD project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Forte AUD project's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects. Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

No high severity issues were discovered.

Medium

[M-01] Beneficiaries.sol#removeBeneficiary - Deleting a beneficiaries could be dangerous as this may not properly delete the `InternalBeneficiary.limiter` mapping(s) which can result in inconsistencies when a new beneficiary uses the same key

Description

The `removeBeneficiary` function is responsible for removing beneficiaries from the contract which makes use of the `delete` keyword; however, for mappings, the `delete` keyword has no effect.

Vulnerability Details

When the contract deletes a struct from the contract, it will not delete the mapping inside of it. The `delete` keyword in Solidity will simply reset all the fields in the struct to its default value (for example: `false` for `bool` and `0` for `uint`). This is the case because mappings are implemented as hash maps and the EVM does not keep track of which keys have been used in the mapping. Therefore, it doesn't know how to "reset" a mapping so when a mapping is deleted from a struct, the mapping within the struct will still retain stale data.

Consider the `removeBeneficiary` function and relative data structures used in the `Beneficiaries.sol` contract in order of hierarchy:

```
function removeBeneficiary(Beneficiaries storage self, address _beneficiary) internal
{

        uint128 key = _getBeneficiaryKey(self, _beneficiary);

        // @audit-issue - mappings may not fully be deleted here

        delete self._beneficiaries[key];

        self._keys.remove(key);

    }
```

`Beneficiaries` struct:

```
struct Beneficiaries {

    LinkedList _keys;

    mapping(uint128 => InternalBeneficiary) _beneficiaries;

    mapping(address => uint128) _addressKeys;

}
```

`InternalBeneficiary` struct:

```
struct InternalBeneficiary {

    address account;

    Limiter limiter;

    uint256 enabledAt;

}
```

`Limiter` struct:

```
struct Limiter {

    uint256 interval;

    uint256 limit;

    LinkedList _keys;

    mapping(uint128 => Operation) _transfers;

}
```

As a result, when a beneficiary is deleted, the limiter `_transfers` mapping may not be fully cleared out which in turn may not fully remove the `Operation` variables for its

respective `uint128`. Considering these values are heavily used in the `Limiter` contract, this may cause unexpected behaviour within the contract.

Impact

Beneficiaries may not fully be deleted which may cause unexpected behaviours around calculating the used limit for a beneficiary or when the limiter contract attempts to filter transfers.

Recommendation

It's recommended that when a beneficiary is deleted, the `removeBeneficiary` function recursively walks the data structure and resets all the relevant mappings and attributes within the limiter.

Status

Resolved

Low

[L-01] LinkedList.sol#Implementation - Potential scaling issue when N grows to large enough numbers which may result in out-of-gas reverts

Description

The `LinkedList` library allows contracts to implement doubly linked list data structures within their smart contracts on chain. There isn't any limit to which `N` can grow to where `N` is the length of the linked list which may result in out-of-gas errors.

Impact

Out-of-gas errors could result in a denial of service condition if `N` grows to a substantially high value.

Recommendation

Consider either implementing the doubly linked list logic off-chain and executing transactions on the results on-chain or implementing limits for how large `N` can get.

Status

Unresolved.

Centralisation

The Forte AUD project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the Forte AUD project seems to have a sound and well tested code base, however our findings need to be resolved in order to achieve security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.